

Лабораторная работа №3

Системы контроля версий. Git

Давид Майкл Фрэнсис

В данном отчёте представлены результаты выполнения лабораторной работы по теме «Системы контроля версий. Git».

Описаны установка и настройка Git и GitHub CLI, создание SSH- и PGP-ключей, настройка подписи коммитов и структуры рабочего каталога курса.

Сделаны выводы о достижении поставленной цели.

Содержание

1	Введение	6
1.1	Актуальность	6
1.2	Цель работы	6
1.3	Задачи	6
1.4	Объект и предмет исследования	7
1.5	Методы исследования	7
1.6	Схема выполнения работы	7
2	Теоретическая часть	9
3	Материалы и методы	11
4	Выполнение лабораторной работы	12
5	Результаты	17
6	Обсуждение результатов	18
7	Заключение	19
	Ответы на контрольные вопросы	20
	Список литературы	23

Список иллюстраций

1.1	Блок-схема выполнения лабораторной работы	8
4.1	Установка Git и GitHub CLI	12
4.2	Настройка имени пользователя, почты, кодировки и SSH-ключа . . .	13
4.3	Генерация PGP-ключа	13
4.4	Добавление PGP-ключа в GitHub (шаг 1)	14
4.5	Добавление PGP-ключа в GitHub (шаг 2)	14
4.6	Добавление PGP-ключа в GitHub (шаг 3)	15
4.7	Настройка автоматической подписи коммитов	15
4.8	Настройка каталога курса	16

Список таблиц

1 Введение

1.1 Актуальность

Системы контроля версий являются неотъемлемым инструментом современной разработки программного обеспечения. Они позволяют отслеживать историю изменений, работать над проектом совместно с другими разработчиками и безопасно возвращаться к предыдущим версиям кода. Git, как наиболее распространённая распределённая система контроля версий, лежит в основе большинства современных рабочих процессов разработки, включая работу с платформой GitHub.

1.2 Цель работы

Изучить идеологию и применение средств контроля версий, освоить умения по работе с Git.

1.3 Задачи

- установить Git и GitHub CLI (gh);
- настроить пользовательские данные и кодировку вывода Git;
- создать SSH-ключ для аутентификации;
- создать PGP-ключ и настроить подпись коммитов;
- добавить PGP-ключ в учётную запись GitHub;

- настроить структуру рабочего каталога курса.

1.4 Объект и предмет исследования

Объектом исследования является система контроля версий Git. Предметом исследования выступает процесс установки, настройки и подготовки Git к безопасной работе с удалённым репозиторием GitHub, включая аутентификацию по SSH и подпись коммитов PGP-ключом.

1.5 Методы исследования

Работа выполнялась практическим методом — путём непосредственной установки и настройки программного обеспечения в среде Linux с использованием утилит командной строки (`dnf`, `git`, `gh`, `gpg`, `ssh-keygen`).

1.6 Схема выполнения работы

На рис. 1.1 представлена общая последовательность действий при выполнении работы.

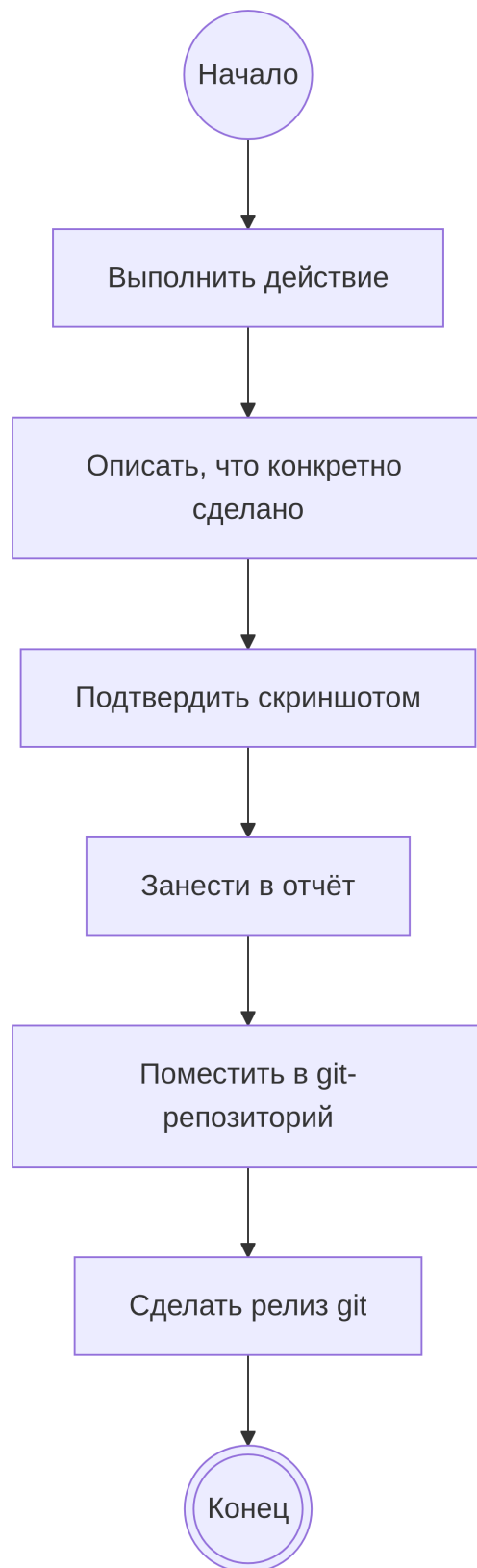


Рисунок 1.1: Блок-схема выполнения лабораторной работы

2 Теоретическая часть

Системы контроля версий (VCS, Version Control System) отслеживают изменения в файлах проекта во времени, позволяя нескольким людям совместно работать над ним, вести историю изменений и возвращаться к предыдущим версиям. Они решают такие задачи, как случайная потеря данных, конфликты при совместном редактировании кода и отслеживание того, кто и когда внёс те или иные изменения [1].

В рамках VCS используются следующие ключевые понятия:

- **хранилище (repository)** — центральное место, где хранятся все версии файлов проекта;
- **фиксация (commit)** — моментальный снимок изменений, сохранённый в репозитории;
- **история (history)** — последовательность коммитов, отражающая прошлые изменения проекта;
- **рабочая копия (working copy)** — локальная версия файлов, с которой непосредственно работает пользователь.

По способу хранения данных различают два вида систем контроля версий:

- **централизованные (CVCS)** — все изменения хранятся в едином центральном репозитории; примеры: Subversion (SVN), Perforce;
- **децентрализованные (DVCS)** — каждый пользователь располагает полной копией репозитория, что позволяет работать автономно, без постоянного подключения к серверу; примеры: Git, Mercurial.

Git, как децентрализованная система контроля версий, решает следующие основные задачи: отслеживание изменений в файлах, организация совместной работы и слияние изменений нескольких участников, ветвление для параллельной разработки независимых функций, а также откат к предыдущим версиям проекта при необходимости.

3 Материалы и методы

Для выполнения работы использовались:

- операционная система семейства Linux (дистрибутив на основе dnf);
- система контроля версий **Git**;
- официальный клиент командной строки **GitHub CLI** (gh);
- утилита **GnuPG** (gpg2) для генерации PGP-ключа и подписи коммитов;
- утилита **ssh-keygen** для генерации SSH-ключа;
- платформа **GitHub** в качестве удалённого хостинга репозиториев.

4 Выполнение лабораторной работы

Первым шагом были установлены Git и GitHub CLI с помощью менеджера пакетов dnf (рис. 4.1).

```
and without modifying the system state.
midavid@vbox:~$ sudo -i
[sudo] password for midavid:
root@vbox:~# dnf install git
Updating and loading repositories:
google-chrome          100% | 4.4 KiB/s | 1.3 KiB | 00m00s
Copr repo for PyCharm owned by phracek 100% | 2.8 KiB/s | 2.1 KiB | 00m01s
RPM Fusion for Fedora 41 - Nonfree - S 100% | 13.0 KiB/s | 8.7 KiB | 00m01s
RPM Fusion for Fedora 41 - Nonfree - N 100% | 34.3 KiB/s | 9.1 KiB | 00m00s
Fedora 41 - x86_64 - Updates          100% | 55.7 KiB/s | 25.1 KiB | 00m00s
google-chrome          100% | 8.4 KiB/s | 3.2 KiB | 00m00s
RPM Fusion for Fedora 41 - Nonfree - N 100% | 31.4 KiB/s | 18.2 KiB | 00m01s
Fedora 41 - x86_64 - Updates          100% | 2.6 MiB/s | 7.2 MiB | 00m03s
Repositories loaded.
Package "git-2.48.1-1.fc41.x86_64" is already installed.

Nothing to do.
root@vbox:~# git config --global Ushie47 "Michael david"
error: key does not contain a section: Ushie47
root@vbox:~# dnf install gh
Updating and loading repositories:
Repositories loaded.
Package Arch Version Repository Size
Installing:
```

Рисунок 4.1: Установка Git и GitHub CLI

Затем были указаны имя пользователя и адрес электронной почты для коммитов, настроен вывод сообщений Git в кодировке UTF-8, включена проверка и подпись коммитов, а также создан SSH-ключ для аутентификации (рис. 4.2).

```

Complete!
root@vbox:~# git config --global user.name "Michael David"
root@vbox:~# git config --global user.email "ukanadaushie@gmail.com"
root@vbox:~# git config --global core.quotepath false
root@vbox:~# git config --global init.defaultBranch master
root@vbox:~# git config --global core.autocrlf input
root@vbox:~# git config --global core.safecrlf warn
root@vbox:~# ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/root/.ssh/id_rsa):
Enter passphrase for "/root/.ssh/id_rsa" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /root/.ssh/id_rsa
Your public key has been saved in /root/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:PqD2qYvtkCIW5BI0DaVHiAJ4pbZmr8RB/YVlouyTDIE root@vbox
The key's randomart image is:
+---[RSA 4096]---+
|==*+. . o      |
|E.*. . =       |
|O+.=+ . .      |
|o+.= o .       |
|.o.= o S       |
|.+.+ o o       |
|o.= + o        |

```

Рисунок 4.2: Настройка имени пользователя, почты, кодировки и SSH-ключа

Далее был сгенерирован PGP-ключ с помощью команды `gpg --full-generate-key` (рис. 4.3).

```

root@vbox:~# gpg --full-generate-key
gpg (GnuPG) 2.4.5; Copyright (C) 2024 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Please select what kind of key you want:
(1) RSA and RSA
(2) DSA and Elgamal
(3) DSA (sign only)
(4) RSA (sign only)
(9) ECC (sign and encrypt) *default*
(10) ECC (sign only)
(14) Existing key from card
Your selection? 1
RSA keys may be between 1024 and 4096 bits long.
What keysize do you want? (3072) 4096
Requested keysize is 4096 bits
Please specify how long the key should be valid.
    0 = key does not expire
    <n> = key expires in n days
    <n>w = key expires in n weeks
    <n>m = key expires in n months

```

Рисунок 4.3: Генерация PGP-ключа

Сгенерированный ключ был добавлен в учётную запись GitHub через настройки профиля (рис. 4.4, рис. 4.5, рис. 4.6).

```

We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
gpg: /root/.gnupg/trustdb.gpg: trustdb created
gpg: directory '/root/.gnupg/openpgp-revocs.d' created
gpg: revocation certificate stored as '/root/.gnupg/openpgp-revocs.d/0E9F96A906BAF43
16C1D8D28E27DFA4588FD67F8.rev'
public and secret key created and signed.

pub   rsa4096 2025-03-02 [SC]
      0E9F96A906BAF4316C1D8D28E27DFA4588FD67F8
uid           Michael <omotole47@gmail.com>
sub   rsa4096 2025-03-02 [E]

root@vbox:~# gpg --list-secret-keys --keyid-format LONG
gpg: checking the trustdb
gpg: marginals needed: 3  completes needed: 1  trust model: pgp
gpg: depth: 0  valid:   1  signed:   0  trust: 0-, 0q, 0n, 0m, 0f, 1u
[keyboxd]
-----
sec   rsa4096/E27DFA4588FD67F8 2025-03-02 [SC]

```

Рисунок 4.4: Добавление PGP-ключа в GitHub (шаг 1)

```

pub   rsa4096 2025-03-02 [SC]
      0E9F96A906BAF4316C1D8D28E27DFA4588FD67F8
uid           Michael <omotole47@gmail.com>
sub   rsa4096 2025-03-02 [E]

root@vbox:~# gpg --list-secret-keys --keyid-format LONG
gpg: checking the trustdb
gpg: marginals needed: 3  completes needed: 1  trust model: pgp
gpg: depth: 0  valid:   1  signed:   0  trust: 0-, 0q, 0n, 0m, 0f, 1u
[keyboxd]
-----
sec   rsa4096/E27DFA4588FD67F8 2025-03-02 [SC]
      0E9F96A906BAF4316C1D8D28E27DFA4588FD67F8
uid           [ultimate] Michael <omotole47@gmail.com>
ssb   rsa4096/29A1F451E7B997DF 2025-03-02 [E]

root@vbox:~# gpg --armor --export <PGP Fingerprint> | xclip -sel clip
-bash: syntax error near unexpected token `|'
root@vbox:~# sudo apt install xclip
sudo: apt: command not found
root@vbox:~# gpg --armor --export E27DFA4588FD67F8
bash: gpg: command not found...
root@vbox:~# gpg --armor --export E27DFA4588FD67F8
-----BEGIN PGP PUBLIC KEY BLOCK-----
mQINRGcf0g8EADfTc6zw@vEx3DdEXV30EHWst176gKuMhSTZu5MdeoEgnToRiAN

```

Рисунок 4.5: Добавление PGP-ключа в GitHub (шаг 2)

GPG keys

[New GPG key](#)

This is a list of GPG keys associated with your account. Remove any keys that you do not recognize.

 **Gpg key**

Email address: omotole47@gmail.com Unverified

Key ID: E27DFA4588FD67F8

Subkeys: 29A1F451E7B997DF

Added on Mar 2, 2025

[Delete](#)

Learn how to [generate a GPG key and add it to your account](#).

Рисунок 4.6: Добавление PGP-ключа в GitHub (шаг 3)

После этого Git был настроен на автоматическую подпись коммитов сгенерированным PGP-ключом:

```
git config --global user.signingkey <PGP Fingerprint>
git config --global commit.gpgsign true
git config --global gpg.program $(which gpg2)
```

Результат настройки показан на рис. 4.7.

```
====END PGP PUBLIC KEY BLOCK====
root@vbox:~# git config --global user.signingkey <E27DFA4588FD67F8>
-bash: syntax error near unexpected token `newline'
root@vbox:~# git config --global user.signingkey <E27DFA4588FD67F8>
-bash: syntax error near unexpected token `newline'
root@vbox:~# git config --global user.signingkey E27DFA4588FD67F8
root@vbox:~# git config --global commit.gpgsign true
root@vbox:~# git config --global gpg.program $(which gpg2)
root@vbox:~# gh auth login
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? SSH
? Upload your SSH public key to your GitHub account? /root/.ssh/id_ed25519.pub
? Title for your SSH key: Ssh key
? How would you like to authenticate GitHub CLI? Login with a web browser

! First copy your one-time code: AE5C-EFF6
Press Enter to open https://github.com/login/device in your browser...
Running Firefox as root in a regular user's session is not supported. ($XAUTHORITY
is /run/user/1000/.mutter-Xwaylandauth.ADVQ22 which is owned by midavid.)
✓ Authentication complete.
- gh config set -h github.com git_protocol ssh
✓ Configured git protocol
! Authentication credentials saved in plain text
✓ Uploaded the SSH key to your GitHub account: /root/.ssh/id_ed25519.pub
✓ Logged in as Ushie47
root@vbox:~# mkdir -p ~/study-2024-2025/"operations systems"
```

Рисунок 4.7: Настройка автоматической подписи коммитов

Заключительным шагом стала настройка структуры рабочего каталога курса (рис. 4.8).

```
root@vbox:~/study-2024-2025_os-intro/operations systems# mkdir os-intro
root@vbox:~/study-2024-2025_os-intro/operations systems# echo os-intro > COURSE
root@vbox:~/study-2024-2025_os-intro/operations systems# git add .
root@vbox:~/study-2024-2025_os-intro/operations systems# cd ~
root@vbox:~# git add .
fatal: not a git repository (or any of the parent directories): .git
root@vbox:~# cd study-2024-2025_os-intro
root@vbox:~/study-2024-2025_os-intro# git add .
root@vbox:~/study-2024-2025_os-intro# git commit -am 'feat(main): make course structure'
midavid@vbox:~$
```

Рисунок 4.8: Настройка каталога курса

5 Результаты

В результате выполнения работы:

- установлены и настроены Git и GitHub CLI;
- заданы пользовательские данные (имя, email) и кодировка вывода UTF-8;
- создан SSH-ключ для аутентификации на GitHub;
- сгенерирован PGP-ключ и добавлен в учётную запись GitHub;
- настроена автоматическая подпись коммитов PGP-ключом;
- подготовлена структура рабочего каталога курса.

6 Обсуждение результатов

Настройка подписи коммитов PGP-ключом обеспечивает возможность проверки подлинности автора изменений — на GitHub такие коммиты отмечаются как «Verified», что повышает доверие к истории репозитория при совместной работе. Использование SSH-ключа вместо пароля для аутентификации соответствует современным требованиям безопасности, поскольку GitHub более не поддерживает аутентификацию по паролю для операций с Git. Выполненная настройка полностью соответствует стандартной практике подготовки рабочего окружения для децентрализованной разработки с использованием Git.

7 Заключение

В ходе лабораторной работы были изучены идеология и применение средств контроля версий. Освоены основные умения по работе с Git:

- создание и клонирование репозитория;
- создание SSH- и PGP-ключей;
- настройка подписи коммитов;
- получение изменений из удалённого репозитория и отправка изменений на GitHub.

Поставленная цель работы достигнута.

Ответы на контрольные вопросы

1. Что такое системы контроля версий (VCS) и для решения каких задач они предназначаются?

Системы контроля версий (VCS) отслеживают изменения в файлах, позволяя нескольким людям совместно работать над проектом, вести историю и возвращаться к предыдущим версиям. Они решают такие проблемы, как случайная потеря данных, конфликты кода и отслеживание изменений.

2. Объясните следующие понятия VCS и их отношения: хранилище, commit, история, рабочая копия.

- Хранилище — центральное место, где хранятся все версии файлов.
- Коммит (фиксация) — моментальный снимок изменений, сохранённых в репозитории.
- История — запись коммитов, отражающая прошлые изменения.
- Рабочая копия — локальная версия файлов, изменяемых пользователем.

3. Что представляют собой и чем отличаются централизованные и децентрализованные VCS? Приведите примеры VCS каждого вида.

- Централизованная система контроля версий (CVCS): все изменения хранятся в едином центральном репозитории. Пример: Subversion (SVN), Perforce.
- Децентрализованная система контроля версий (DVCS): у каждого пользователя есть полная копия репозитория, что позволяет работать в автономном режиме. Пример: Git, Mercurial.

4. Действия VCS при работе с хранилищем в одиночку

- Инициализировать репозиторий.
- Вносить изменения и фиксировать их.
- Просматривать историю и при необходимости возвращаться к ней.

5. Работа с VCS при совместном использовании хранилища

- Клонировать репозиторий или получить последнюю версию.
- Вносить изменения и фиксировать их локально.
- Отправлять изменения в общий репозиторий.
- При необходимости разрешать конфликты.

6. Основные задачи, решаемые Git

- Отслеживание изменений.
- Совместная работа и слияние.
- Ветвление для параллельной разработки.
- Откат версии.

7. Роли участников при работе с Git

- Разработчики (developers) — пишут и фиксируют код.
- Сопровождающие (maintainers) — проверяют и объединяют изменения.
- Релиз-менеджеры (release managers) — управляют версиями и релизами.

8. Использование локальных и удалённых репозиториев

Локальный репозиторий:

```
git init
git add .
git commit -m "Initial commit"
```

Удалённый репозиторий:

```
git remote add origin <repo_url>  
git push origin main  
git pull origin main
```

9. Ветви и их назначение

Ветки позволяют вести параллельную разработку, не затрагивая основную кодовую базу:

```
git branch feature-branch  
git checkout feature-branch
```

10. Игнорирование файлов в Git

```
node_modules/  
*.log
```

Файл `.gitignore` с таким содержимым предотвращает попадание ненужных файлов в репозиторий.

Список литературы

1. *Chacon S., Straub B.* Pro Git. — 2nd ed. — Apress, 2014. — ISBN 978-1484200773. — Free online edition: <https://git-scm.com/book/en/v2>.